

Does Live Browser Context Improve LLM-Assisted End-to-End Test Script Repair? An Empirical Study of Playwright MCP

Rifa Indra Setiawan¹, Divi Galih Prasetyo Putri¹,

¹Department of Electronic and Informatics Engineering, Universitas Gadjah Mada

Correspondence: divi.galih@ugm.ac.id

Abstract

Automated End-to-End test scripts are inherently fragile. Modifications to the User Interface (UI) cause automation test script failures, imposing significant maintenance overhead. Automated test script repair using Large Language Model (LLM) has shown promise, but existing approaches operate without real-time access to the application under test. To the best of our knowledge, no prior controlled study has compared LLM-based test-script repair with and without MCP-mediated live browser access while keeping the remaining repair settings consistent. Through a controlled experiment on a web application, we inject 30 failure scenarios across two categories of UI modifications (locator changes and DOM structural changes) and evaluate two LLM models (GPT 5.0 Mini and Claude Haiku 4.5) with two conditions (with and without MCP access). Configurations with MCP achieve a correct patch rate of 65%, compared with 25% without MCP. The plausible patch rate reached 98% (vs. 87%), the failure rate dropped from 13.3% to 1.7%, and MTTR decreased by 31 seconds. These results suggest that providing agents with MCP-mediated live browser access primarily enhances repair *correctness* rather than merely increasing re-execution success rates.

1 Introduction

Automated End-to-End (E2E) testing has become an important part of modern web development, particularly within agile and continuous integration/continuous delivery (CI/CD) workflows (Niu et al., 2025). By executing test cases automatically through frameworks such as Playwright and Selenium, teams can validate application behavior across full user flows with reduced manual effort (Melyawati et al., 2024). However, a persistent challenge undermines the long-term value of automation testing which is **test script fragility**. When front-end interfaces evolve, existing test

scripts break and require significant maintenance effort (Samad et al., 2021). Awad et al. (2024) report that testers can spend approximately 40% of their time updating and maintaining test suites as applications evolve. Prior empirical studies have consistently identified locator changes and DOM structural modifications as the most dominant causes of test script failure across multiple frameworks (Parry et al., 2022; Di Meglio et al., 2026). Even minor changes, such as renaming a CSS class or altering an element’s position in the DOM hierarchy, can cause complete script failure (Nass et al., 2023). These challenges are not transient. UI test automation will continue to face long-term difficulties in script maintenance, stability, and adaptation to interface changes (Nass et al., 2021).

To reduce this maintenance effort, automated test script repair mechanisms have been proposed to detect, diagnose, and repair broken test scripts with minimal manual intervention (Saarathy et al., 2024). Existing approaches rely on techniques such as heuristic-based locator matching, semantic information from previous test executions, and more recently, large language model (LLM)-based autonomous agents to recover failed test scripts (Grover and Sepuri, 2025; Qi et al., 2023; Bouzenia et al., 2025). Despite these advances, existing test repair approaches increasingly incorporate information from the current state of the application under test. Heuristic-based systems such as Healenium use historical locator information together with runtime information from the updated interface to identify replacement elements (Grover and Sepuri, 2025). Recent AI-assisted approaches may also use failure artifacts or information from the current user interface to guide repair decisions (Biswas, 2026). However, these approaches are typically evaluated as complete repair systems, making it difficult to determine the specific contribution of live and interactive browser context. Consequently,

it remains unclear whether giving an LLM-based repair agent direct access to the current browser state actually improves the semantic correctness of generated repairs, rather than merely increasing the likelihood that the modified test script passes execution. This distinction is important because a passing repair may weaken, remove, or circumvent the original verification logic, thereby masking genuine application regressions.

In this paper, we investigate whether MCP-mediated live browser access improves the quality of LLM-assisted test-script repair compared with artifact-based repair alone. MCP is a standardized protocol for exposing external tools and contextual information to AI applications (Sudakara, 2025). In this study, we use **Playwright** as the automation framework for executing and validating end-to-end web tests, together with the Playwright MCP Server. This implementation exposes Playwright’s browser interaction capabilities to the LLM-based repair agent, allowing it to navigate web pages, obtain structured accessibility snapshots, interact with interface elements, and observe the current browser state during the repair process, as illustrated in Figure 1 (Hou et al., 2026).

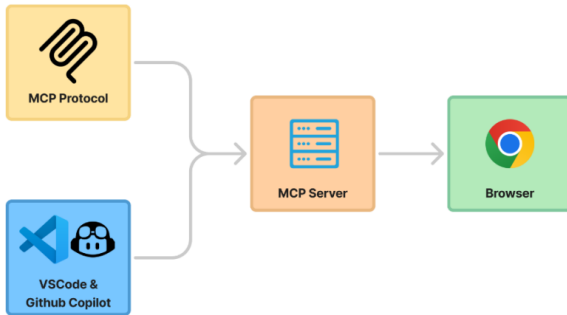


Figure 1: Implementation overview of Playwright MCP with an AI agent. The MCP server bridges the static LLM knowledge with browser context tools, allowing the agent to observe the live DOM state during self-healing.

We conduct a controlled experiment on a web application by injecting 30 failure scenarios across two categories, 15 scenarios related to locator changes and 15 scenarios related to DOM structural changes. Each scenario is tested under four configurations that combine two LLM models (GPT 5.0 Mini and Claude Haiku 4.5) with two conditions (with and without MCP access). Following the Automated Program Repair (APR) evaluation framework (Dikici and Bilgin, 2025), we measure three metrics: *plausible patch rate* (proportion of

repairs that restore test execution), *correct patch rate* (proportion of repairs that match manually constructed ground truth), and *Mean Time to Repair* (MTTR).

2 Related Work

Several approaches have been proposed to automatically repair broken test scripts, broadly categorizable into heuristic-based, semantic-based, and LLM-based methods. Grover and Sepuri (2025) compare traditional Selenium frameworks with the Healenium self-healing stack, which uses attribute-similarity heuristics to find replacement locators when the original locator fails. While Healenium significantly reduces failures caused by locator changes, it relies on syntactic matching without understanding the functional purpose of the target element. This creates a risk of *regression masking*, where a repaired script passes execution but the chosen element is functionally incorrect (Biswas, 2026). Similarly, Kim and Kouatly (2024) integrate Selenium with the ChatGPT API for self-healing, demonstrating improved maintenance efficiency, though the approach still depends on static context for repair decisions. Qi et al. (2023) propose Semter, which captures semantic information from test executions on an earlier application version and uses semantic similarity to locate target elements in updated versions. Semter achieves an average repair ratio of 84% across six web applications and can handle workflow changes caused by page additions or removals. However, its effectiveness depends on the quality of semantic information from initial executions and degrades when interface changes diverge substantially from the original version. Pham et al. (2022) review AI-augmented E2E testing tools and find that existing self-healing features remain limited to element-level locator matching, lacking the ability to handle complex functional or structural changes. More recently, LLM-based agents have shown promise for automated repair tasks. Bouzenia et al. (2025) introduce RepairAgent, an autonomous LLM-based agent for Automated Program Repair (APR) that successfully fixes 164 bugs on the Defects4J dataset through iterative reasoning. Although RepairAgent targets production code rather than test scripts, its APR framework provides a rigorous evaluation methodology applicable to code repair assessment. Most closely related to our work, Biswas (2026) implements an AI-assisted self-healing agent (Play-

wright Healer Agent) for E2E web testing, achieving a 73% reduction in false positives, 45% reduction in MTTR, and 91.3% test reliability. However, this approach performs inference solely from the LLM’s internal knowledge and static error logs, without the ability to access or verify the live page state, leaving it vulnerable to regression masking. A common limitation across all prior automated script repair approaches is the absence of real-time interaction with the application under test during the repair process. MCP addresses this gap by standardizing how AI systems integrate with external tools, data sources, and environments (Hou et al., 2026). MCP follows a client-server architecture where an MCP host (e.g., an IDE with an AI agent) communicates with MCP servers that expose domain-specific capabilities (Sudakara, 2025). LLM-based agents, which combine LLM reasoning with tool use, memory, and planning modules (Chowa et al., 2026), are particularly well-suited to leverage MCP’s capabilities. He et al. (2025) demonstrate that modern LLMs’ planning and reasoning abilities enable agents to handle complex tasks including code generation, testing, and script maintenance autonomously. However, LLMs are fundamentally reasoning engines rather than data repositories. LLM knowledge is static and they cannot directly observe live application state (Lee et al., 2024). MCP bridges this gap by providing grounded, real-time context that enables more informed repair decisions. Despite the conceptual promise of this integration, no prior work has empirically evaluated MCP’s contribution to automated script repair in automated testing. Our study addresses this gap through a controlled comparison of LLM-based repair with and without MCP-mediated live browser access. The experiment therefore evaluates the practical contribution of adding MCP-enabled live interaction to an otherwise comparable agent workflow.

3 Methodology

3.1 System Under Test

We conduct our experiments on a locally deployed web-based information system for managing teaching assistant recruitment. The application is developed in PHP using the Laravel framework. We focus on the student role, which involves a sequence of functions, including completing a profile, uploading academic transcripts, browsing available positions, registering for a vacancy, and managing

offers. Such interactions are representative of the user-oriented workflows typically evaluated in end-to-end web testing, where tests navigate through the application, perform user actions, and verify observable outcomes. Full access to the source code and local deployment environment enables us to introduce controlled interface changes for fault injection while preserving the application’s underlying business functionality.

3.2 Test Design

We design 15 End-to-End (E2E) test cases (TC01–TC15) using Playwright with TypeScript, organized across five functional areas: profile management (F1=4 TCs), transcript upload (F2=3 TCs), vacancy registration (F3=3 TCs), registration verification (F4=3 TCs), and offer management (F5=2 TCs). All test scripts follow the Page Object Model (POM) pattern, separating locator definitions, page interaction logic, and test specifications into distinct layers. Baseline scripts are validated through three consecutive executions with zero failures before experimentation.

3.3 Fault Injection

We inject 30 failure scenarios across two categories: Locator changes and DOM structural changes. The scenarios were designed to represent changes that commonly occur during web application evolution (Nass et al., 2023; Di Meglio et al., 2026). Examples of failure scenarios are shown in Table 1.

Locator changes (LC01–LC15). These scenarios modify element attributes that test scripts use for identification, such as changing data-testid formats (e.g., kebab-case to camelCase), renaming CSS classes, altering button text that affects accessible names, or modifying XPath-dependent text content. Each injection targets a different locator strategy to evaluate self-healing across varying fragility levels. The purpose was to evaluate whether the repair agent merely updates a broken locator value or identifies a more stable locator strategy when one is available.

DOM structural changes (DC01–DC15). These scenarios modify the HTML hierarchy or introduce new interface elements, such as wrapping elements in additional containers, adding confirmation steps before form submission, introducing input validation, or disabling buttons based on application state. DOM changes are generally more complex as they

ID	Type	Changes	Impact
LC03	LC	testId format: kebab-case → camel- Case	Selector miss
LC08	LC	CSS class renamed	Class mismatch
LC10	LC	Button text changed	Name mismatch
DC01	DC	Element wrapped in new <div>	Hierarchy break
DC10	DC	Confirmation step added	Missing step
DC13	DC	Button disabled on state	Logic change

Table 1: Representative examples from each category. LC represents locator changes and DC represents DOM changes. All injections are designed to reflect realistic front-end changes encountered during application maintenance, not artificial scenarios targeting specific repair strategies.

may require not only locator updates but also modifications to interaction logic.

3.4 Ground Truth Construction

For each of the 30 injection scenarios, we manually write the correct repair following Playwright’s locator stability hierarchy: testId (most stable) > role with accessible name > label > CSS/XPath (most fragile). Repairs follow two patterns: (1) *value update* which retains the baseline locator strategy and adjusting only the changed value when the strategy is already stable, or (2) *strategy upgrade* by replacing a fragile locator strategy with a more stable one. This produces 30 ground truth repairs, each documented as a unified diff file in a version-controlled repository.¹

3.5 Experimental Configurations

We evaluate four configurations that cross two independent variables:

MCP condition– With MCP: the AI agent has access to Playwright MCP Server, enabling real-time browser navigation, accessibility snapshots,

¹Repository: <https://github.com/self-healing-automation/self-healing-automation-MCP>

and live DOM interaction. **Without MCP:** the AI agent operates solely from test scripts, failure screenshots, error logs, and its internal knowledge. This represents the current state of LLM-based self-healing approaches without adding any external capabilities.

LLM model– GPT 5.0 Mini: a thinking-architecture model from the GPT-5 family with a 400K token context window, optimized for deep reasoning at reduced computational cost. **Claude Haiku 4.5:** a speed-optimized model with 4–5× lower latency than Claude Sonnet 4.5, designed for high throughput with competitive accuracy.

Config	MCP	Model
GPT + MCP	✓	GPT 5 Mini
GPT – MCP	×	GPT 5 Mini
Claude + MCP	✓	Haiku 4.5
Claude – MCP	×	Haiku 4.5

Table 2: Experimental configurations. Each configuration is tested on all 30 failure test scenarios (15 LC + 15 DC).

Both models are accessed through GitHub Copilot Chat in Visual Studio Code operating in *agent mode*, which enables autonomous file editing, command execution, and tool invocation. Table 2 summarizes the experimental design.

3.6 Prompting Strategy

To promote consistency across experimental sessions, we designed two standardized prompt variants corresponding to the MCP and non-MCP conditions. Both variants assigned the agent the same role as a QA automation engineer and specified an identical repair objective, technical context, Run–Analyze–Investigate–Fix–Verify workflow, and hard constraints. These constraints prohibited the agent from removing or weakening assertions, skipping test steps, introducing arbitrary waits, using forced interactions, or modifying authentication fixtures. No scenario-specific repair hints were included, and the same prompt variant was reused across all sessions within each condition.

Both conditions allowed the agent to execute and verify tests through Playwright and inspect the resulting error output, stack traces, and failure screenshots. They differed only in the investigation capabilities available to the agent. In the non-

MCP condition, browser information was mediated by the prescribed test execution and its generated artifacts. In the MCP condition, the agent additionally had direct access to the running browser through Playwright MCP, enabling agent-directed navigation, accessibility snapshots, and interface interaction during diagnosis.

Before each session, the test scripts were restored to their baseline state using version control, and a new agent session was initialized. The fault-injected test was first executed to confirm the expected failure. The agent then performed the repair workflow and was allowed up to three verification cycles. A session was classified as failed when the test remained unsuccessful after the third cycle. The complete prompt variants are provided in the appendix.

3.7 Evaluation Metrics

Following the APR evaluation framework (Dikici and Bilgin, 2025; Bouzenia et al., 2025), we evaluate repairs using three metrics:

Plausible patch rate. The proportion of scenarios where the repaired script executes successfully:

$$\text{Plausible Rate} = \frac{|\text{Passing Scenarios}|}{|\text{Total Scenarios}|} \times 100\% \quad (1)$$

Correct patch rate. The proportion of repairs that match the ground truth, either identically or through functional equivalence as determined by manual evaluation (Le et al., 2019):

$$\text{Correct Rate} = \frac{|\text{Ground Truth Matches}|}{|\text{Total Scenarios}|} \times 100\% \quad (2)$$

Mean Time to Repair (MTTR). The average time from the start of the agent’s analysis to successful verification:

$$\text{MTTR} = \frac{\sum \text{Repair Time}}{|\text{Repaired Scenarios}|} \quad (3)$$

Each repair is classified into one of four categories: **Correct (C)** indicates that the repaired test passes and matches ground truth; **Plausible Incorrect (PI)** indicates that the repaired test passed but differs from ground truth; **Regression Masking (RM)** indicates that the test passes because assertions or test logic have been removed, weakened, or bypassed; and **Fail (F)** indicates that the test does not pass after repair. Figure 2 illustrates these four

outcomes in the context of the sequential End-to-End execution flow. Each scenario is executed once per configuration due to token quota constraints on the AI agent platform, yielding 120 total sessions. We acknowledge this limitation and discuss its implications in Section 5.

4 Results and Discussion

Prior to evaluating repair performance, all 15 baseline test cases were executed in three consecutive validation runs and passed without any observed flaky behavior. Each of the 30 fault-injection scenarios subsequently triggered the expected test failure, confirming both the stability of the baseline test suite and the effectiveness of the injected changes.

Table 3 presents the aggregate results for each configuration across all 30 scenarios. Configurations with MCP consistently outperform their non-MCP counterparts on both quality metrics.

Config	Plaus.	Correct	Fail	MTTR
GPT – MCP	83%	30%	17%	3:50
Claude – MCP	90%	20%	10%	1:34
GPT + MCP	97%	70%	3%	2:39
Claude + MCP	100%	60%	0%	1:39

Table 3: Per-configuration results across 30 scenarios. Plaus. = plausible patch rate; Correct = correct patch rate; Fail = failure rate; MTTR in min:sec.

The largest observed difference between the condition "With MCP" and "Without-MCP" is in the correct patch rate. GPT improves the correct patch rate from 30% to 70% (+40pp) and Claude from 20% to 60% (+40pp). Plausible rate improvements are comparatively modest (GPT: +14pp; Claude: +10pp), indicating that the largest observed benefit of the MCP-enabled condition lies in repair correctness rather than merely in making scripts pass.

Claude + MCP achieves the highest plausible rate (100%) but a lower correct rate (60%) than GPT + MCP (70%). In the observed repair traces, Claude Haiku 4.5 more frequently applied an initially viable fix, whereas GPT-5 mini more frequently inspected alternative locator strategies before applying a patch. These observations may help explain the measured speed–correctness trade-off, although the limited number of runs prevents generalizing these behavioral differences to the models as a whole. Figure 3 illustrates this trade-off between repair speed and correctness across all configurations.

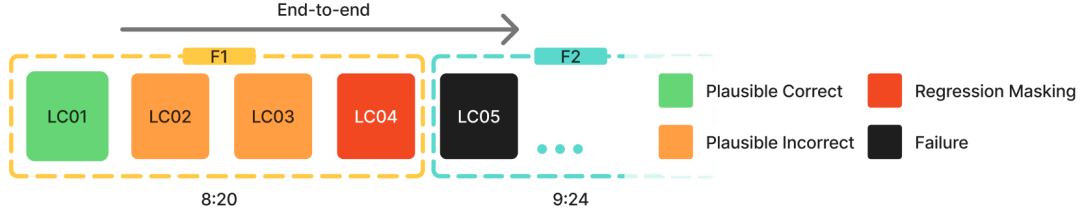


Figure 2: Visualization of the End-to-End self-healing execution flow.

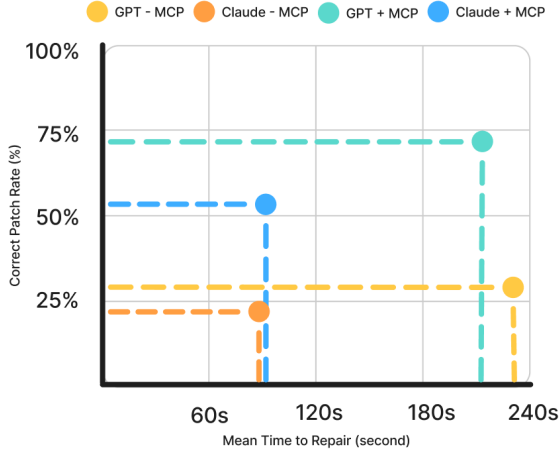


Figure 3: Efficiency diagram plotting MTTR against Correct Patch Rate.

Table 4 presents MTTR disaggregated by injection type and MCP condition.

Condition	Type	Total Time	MTTR
Without MCP	LC	61:51	2:03
With MCP	LC	54:25	1:48
Without MCP	DC	100:49	3:21
With MCP	DC	75:10	2:30

Table 4: MTTR by injection type and MCP condition (min:sec). LC = locator change; DC = DOM change.

DOM changes require substantially longer repair times than locator changes across both conditions, reflecting their greater complexity. Total repair time was 7 minutes 26 seconds lower for locator changes and 25 minutes 39 seconds lower for DOM changes in the MCP-enabled condition. Notably, Claude without MCP records the lowest individual MTTR (1:34 overall), as it skips browser interaction and selects fixes more aggressively. However, this speed comes at the cost of the lowest correct rate (20%) and highest regression masking rate, illustrating that repair efficiency must be evaluated jointly with repair quality.

With MCP. Agents navigate the SUT autonomously, take accessibility snapshots of relevant pages, and use live DOM data to inform locator selection. When navigating to an incorrect page, agents self-correct by recognizing the mismatch between expected and observed page content. For example, in scenario LC08, GPT + MCP first identifies that the CSS class has changed, then discovers a more stable data-testid attribute on the same element, and selects the testId strategy—matching the ground truth. This two-stage verification process (identify change → evaluate alternatives) is characteristic of the thinking-architecture model as we can see from Figure 4.

Without MCP. Agents rely entirely on static error logs, failure screenshots, and their internal knowledge. Without the ability to verify assumptions against the live DOM, agents tend to make more aggressive modifications, sometimes altering valid test logic or removing assertions to make scripts pass. In scenario LC03, Claude without MCP removes the file name verification step entirely after failing to locate the element—producing regression masking despite explicit prompt instructions prohibiting assertion removal. This pattern explains the higher regression masking rate in non-MCP configurations (7 instances vs. 1 with MCP).

Model differences. The two models exhibited different observed trade-offs between repair time and correctness. Claude Haiku 4.5 generally completed repairs faster, whereas GPT-5 mini achieved a higher correct patch rate in both conditions. In the inspected repair traces, GPT-5 mini more frequently examined alternative locator strategies before applying a patch, while Claude Haiku 4.5 more often applied an initial viable repair directly. Both models showed the same 40-percentage-point increase in correct patch rate when MCP was added, reaching 70% for GPT-5 mini and 60% for Claude Haiku 4.5.

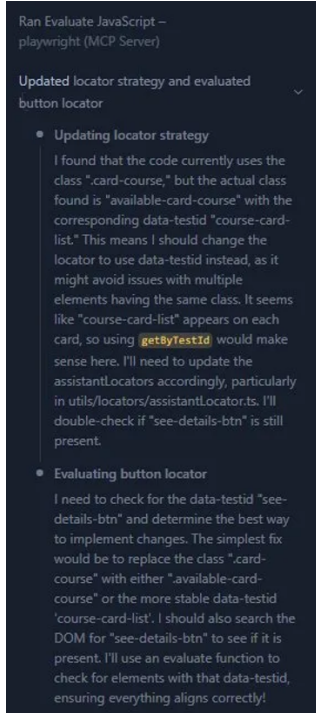


Figure 4: Qualitative example of GPT + MCP on LC08. The agent uses live DOM context to identify a stable data-testid alternative instead of only updating the broken CSS class.

5 Conclusion

We present an empirical evaluation of MCP-mediated live browser contribution to AI-agent-based self-healing in automated End-to-End testing. Through a controlled experiment with 120 sessions across 30 fault injection scenarios and four configurations, we demonstrate that providing AI agents with real-time browser context through MCP substantially contributes to improving repair quality.

Our key findings are twofold. First, the largest difference between conditions occurs in repair correctness: the correct patch rate increases from 25% to 65%, whereas the plausible patch rate increases by 11 percentage points. Second, the MCP-enabled condition also shows a lower failure rate (2% vs. 13%) and an average MTTR that is 31 seconds shorter, with the largest time savings on complex DOM structural changes (25 minutes 39 seconds total).

These results suggest that the integration of structured environmental context through protocols like MCP represents a promising direction for improving the reliability of AI-driven software maintenance tools. Future work should extend this evaluation to multiple applications with diverse architectures, incorporate additional failure types (e.g.,

business logic changes, flaky failures), add experimental repetitions with statistical significance testing, and explore the interaction between MCP and higher-capability LLM models.

Limitations

Our study has several limitations that should be considered when interpreting the results. The observed advantage of MCP is limited to a single locally deployed web application and two interface-related failure types: locator and DOM-structure changes. Its effect may differ for business-logic changes, API modifications, or non-deterministic failures. Moreover, each scenario was executed once per configuration due to GitHub Copilot quota and session limits; therefore, the results reflect single-run outcomes and do not capture run-to-run LLM variability or support statistical significance testing. An important limitation concerns the interpretation of the MCP comparison. The MCP-enabled condition provides the agent with direct access to live application information through browser navigation, accessibility snapshots, and interface interaction, whereas the non-MCP condition relies on information available through test execution artifacts. Therefore, the observed differences cannot be attributed specifically to the structure of the MCP protocol itself. An additional control condition providing equivalent browser information without MCP would be required to disentangle these factors.

Our work is conducted using Playwright MCP Server (v0.0.7) and GitHub Copilot Chat (v0.37.9) that are actively developed tools. The behavior may change in future versions. Our results should be interpreted in the context of the specific versions used during experimentation.

References

- Abdallah Awad, Mahmoud H. Qutqut, Ali Ahmed, Fatima Al-Haj, and Fadi Almasalha. 2024. [Artificial Intelligence Role in Software Automation Testing](#). In *2024 International Conference on Decision Aid Sciences and Applications (DASA)*, pages 1–6, Manama, Bahrain. IEEE.
- Saumen Biswas. 2026. Enhancing End-to-End Test Stability Through AI-Assisted Self-Healing: A Case Study of Playwright Healer Agent Implementation. *International Journal of Engineering & Scientific Research*, 14(01):1–12.
- Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. [RepairAgent: An Autonomous,](#)

- LLM-Based Agent for Program Repair. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 2188–2200, Ottawa, ON, Canada. IEEE.
- Sadia Sultana Chowdhury, Riasad Alvi, Subhey Sadi Rahman, Md Abdur Rahman, Mohaimenul Azam Khan Raiaan, Md Rafiqul Islam, Mukhtar Hussain, and Sami Azam. 2026. *From language to action: A review of large language models as autonomous agents and tool users*. *Artificial Intelligence Review*, 59(2):71.
- Sergio Di Meglio, Luigi Libero Lucio Starace, Valeria Pontillo, Ruben Opdebeeck, Coen De Roover, and Sergio Di Martino. 2026. *Investigating the adoption and maintenance of web GUI testing: Insights from GitHub repositories*. *Information and Software Technology*, 189:107928.
- Sena Dikici and Turgay Tugay Bilgin. 2025. *Advancements in automated program repair: A comprehensive review*. *Knowledge and Information Systems*, 67(6):4737–4783.
- Savi Grover and Shruthi Sepuri. 2025. SELF-HEALING WEB AUTOMATION: AN EMPIRICAL COMPARISON OF TRADITIONAL SELENIUM FRAMEWORKS AND HEALENIUM STACK. *International Journal of Applied Mathematics*, 38(12).
- Junda He, Christoph Treude, and David Lo. 2025. *LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead*. *ACM Transactions on Software Engineering and Methodology*, 34(5):1–30.
- Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2026. *Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions*. *ACM Transactions on Software Engineering and Methodology*, page 3796519.
- Yunjin Kim and Rand Kouatly. 2024. *Self-healing automation testing with Selenium and ChatGPT API integration to improve the efficiency of the maintenance*. In *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, pages 339–344, Dubai, United Arab Emirates. IEEE.
- Xuan-Bach D. Le, Lingfeng Bao, David Lo, Xin Xia, Shanping Li, and Corina Pasareanu. 2019. *On reliability of patch correctness assessment*. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 524–535.
- Hyunji Lee, Doyoung Kim, Kyoung-Woon On, and Minjoon Seo. 2024. *How Well Do Large Language Models Truly Ground?* *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 1:2437–2465.
- Ni Luh Putu Melyawati, I Made Dwi Putra Asana, Ni Nyoman Suardiati Putri, Ketut Jaya Atmaja, and I Gede Iwan Sudipa. 2024. *Comparison of Automation Testing On Card Printer Project Using Playwright And Selenium Tools*. *Journal of Computer Networks, Architecture and High Performance Computing*, 6(3):1309–1320.
- Michel Nass, Emil Alégroth, and Robert Feldt. 2021. *Why many challenges with GUI test automation (will) remain*. *Information and Software Technology*, 138:106625.
- Michel Nass, Emil Alégroth, Robert Feldt, Maurizio Leotta, and Filippo Ricca. 2023. *Similarity-based Web Element Localization for Robust Test Automation*. *ACM Trans. Softw. Eng. Methodol.*, 32(3):75:1–75:30.
- Yingbei Niu, Soo See Chai, Kok Luong Goh, Kim On Chin, and Emily Sing Kiang Siew. 2025. *Continuous Integration and Continuous Delivery Based on ARM Cortex-M4 Embedded Software Automation Testing*. *Journal of Advanced Research in Computing and Applications*, 39(1):146–163.
- Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2022. *A Survey of Flaky Tests*. *ACM Transactions on Software Engineering and Methodology*, 31(1):1–74.
- Phuoc Pham, Vu Nguyen, and Tien Nguyen. 2022. *A Review of AI-augmented End-to-End Test Automation Tools*. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, pages 1–4, Rochester MI USA. ACM.
- Xiaofang Qi, Xiang Qian, and Yanhui Li. 2023. *Semantic Test Repair for Web Applications*. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1190–1202, San Francisco CA USA. ACM.
- Sutharsan Saarathy, Suresh Bathrachalam, and Bharath Rajendran. 2024. *Self-Healing Test Automation Framework using AI and ML*. *International Journal of Strategic Management*, 3(3):45–77.
- Abdus Samad, Tabrez Nafis, Shibli Rahmani, and Shahab Saquib Sohail. 2021. *A Cognitive Approach in Software Automation Testing*. *SSRN Electronic Journal*.
- Baradwaj Bandi Sudakara. 2025. *Leveraging MCP servers for context-aware playwright automation in cloud environments*. *Journal of Emerging Engineering Technologies*, 1(1):13–18.

A Appendix


```

1 ---
2 tools: ['playwright']
3 mode: 'agent'
4 ---
5
6 # Role
7 You are a QA automation engineer fixing one failing Playwright test.
8
9 # Context
10 Playwright Test (TypeScript), Page Object Model:
11 - `tests/` - spec files
12 - `pages/` - page classes
13 - `utils/` - locators
14 - `fixtures/` - setup (do not modify)
15
16 SUT: AssistLab at http://localhost:3000, pre-authenticated via fixtures.
17
18 # Investigation Approach
19 You have access to Playwright MCP browser tools. Use them to inspect
20 the running application directly – navigate to the relevant page,
21 capture accessibility snapshots, and read the actual DOM state before
22 proposing any fix. Runtime observation of the live application is
23 the primary source of evidence for this diagnosis. Also run the test
24 from the terminal as a QA engineer would. If a tool call fails with
25 a snapshot staleness error, capture a new snapshot and retry the
26 interaction.
27
28 # Task
29 Diagnose and fix one failing test in `{SPEC_PATH}`. Work in phases,
30 in order. Maximum 3 fix-verify cycles.
31
32 1. Run – execute `npx playwright test {SPEC_PATH}` and record the
33 exact error and failing step.
34 2. Analyze – read the spec and its dependencies; form an initial
35 hypothesis from the error.
36 3. Investigate – navigate the browser to the relevant page and
37 capture a snapshot. Compare the actual DOM state (elements,
38 attributes, data-testid) against what the locators in the code
39 expect. Report what you observed.
40 4. Fix – apply the smallest change that restores correctness,
41 grounded in the snapshot evidence.
42 5. Verify – re-run the failing test. If it still fails, return to
43 Investigate with new information. If the same approach fails
44 twice in a row, abandon that hypothesis and explore a different
45 angle – re-read the spec, check adjacent locators in the same
46 file, or inspect unrelated page sections that might have changed.
47
48 # Hard Rules
49 Violating any of these is unacceptable even if the test passes:
50 - No weakening or removing assertions
51 - No waitForTimeout or arbitrary sleeps
52 - No { force: true }
53 - No .skip, .only, or commenting out steps
54 - No changes to fixtures/ or auth setup

```

Figure 5: Prompt for test script repair using LLM and MCP

```

1 ---
2 mode: 'agent'
3 ---
4
5 # Role
6 You are a QA automation engineer fixing one failing Playwright test.
7
8 # Context
9 Playwright Test (TypeScript), Page Object Model:
10 - `tests/` - spec files
11 - `pages/` - page classes
12 - `utils/` - locators
13 - `fixtures/` - setup (do not modify)
14
15 SUT: AssistLab at http://localhost:3000, pre-authenticated via fixtures.
16
17 # Investigation Approach
18 You do not have browser automation tools in this session. Diagnose
19 the failure through static code analysis – trace through the spec,
20 page classes, and locator definitions, and reason carefully from
21 the error output. Read the files thoroughly: the specific locator
22 referenced in the error, the page class method that uses it, the
23 spec step that calls the method. Your diagnosis must be grounded
24 in concrete evidence from the code and terminal output. Also run
25 the test from the terminal as a QA engineer would, and read the
26 stack trace in full.
27
28 # Task
29 Diagnose and fix one failing test in `{SPEC_PATH}`. Work in phases,
30 in order. Maximum 3 fix-verify cycles.
31
32 1. Run – execute `npx playwright test {SPEC_PATH}` and record the
33 exact error and failing step.
34 2. Analyze – read the spec and its dependencies; form an initial
35 hypothesis from the error.
36 3. Investigate – navigate the browser to the relevant page and
37 capture a snapshot. Compare the actual DOM state (elements,
38 attributes, data-testid) against what the locators in the code
39 expect. Report what you observed.
40 4. Fix – apply the smallest change that restores correctness,
41 grounded in the snapshot evidence.
42 5. Verify – re-run the failing test. If it still fails, return to
43 Investigate with new information. If the same approach fails
44 twice in a row, abandon that hypothesis and explore a different
45 angle – re-read the spec, check adjacent locators in the same
46 file, or inspect unrelated page sections that might have changed.
47
48 # Hard Rules
49 Violating any of these is unacceptable even if the test passes:
50 - No weakening or removing assertions
51 - No waitForTimeout or arbitrary sleeps
52 - No { force: true }
53 - No .skip, .only, or commenting out steps
54 - No changes to fixtures/ or auth setup

```

Figure 6: Prompt for test script repair using LLM only (without MCP)